# Conception du projet IMT Arena

**Groupe :** Quentin DOUILLY, Lilou CHÂTELAIN & Arthur BEIGNON

Dans ce projet, nous avons séparé le sujet en différentes parties : les équipes et les personnes(joueurs, arbitres, coach), les jeux, les tournois et les matchs. Cette organisation nous permet de garder les règles du tournoi dans les classes concernées.

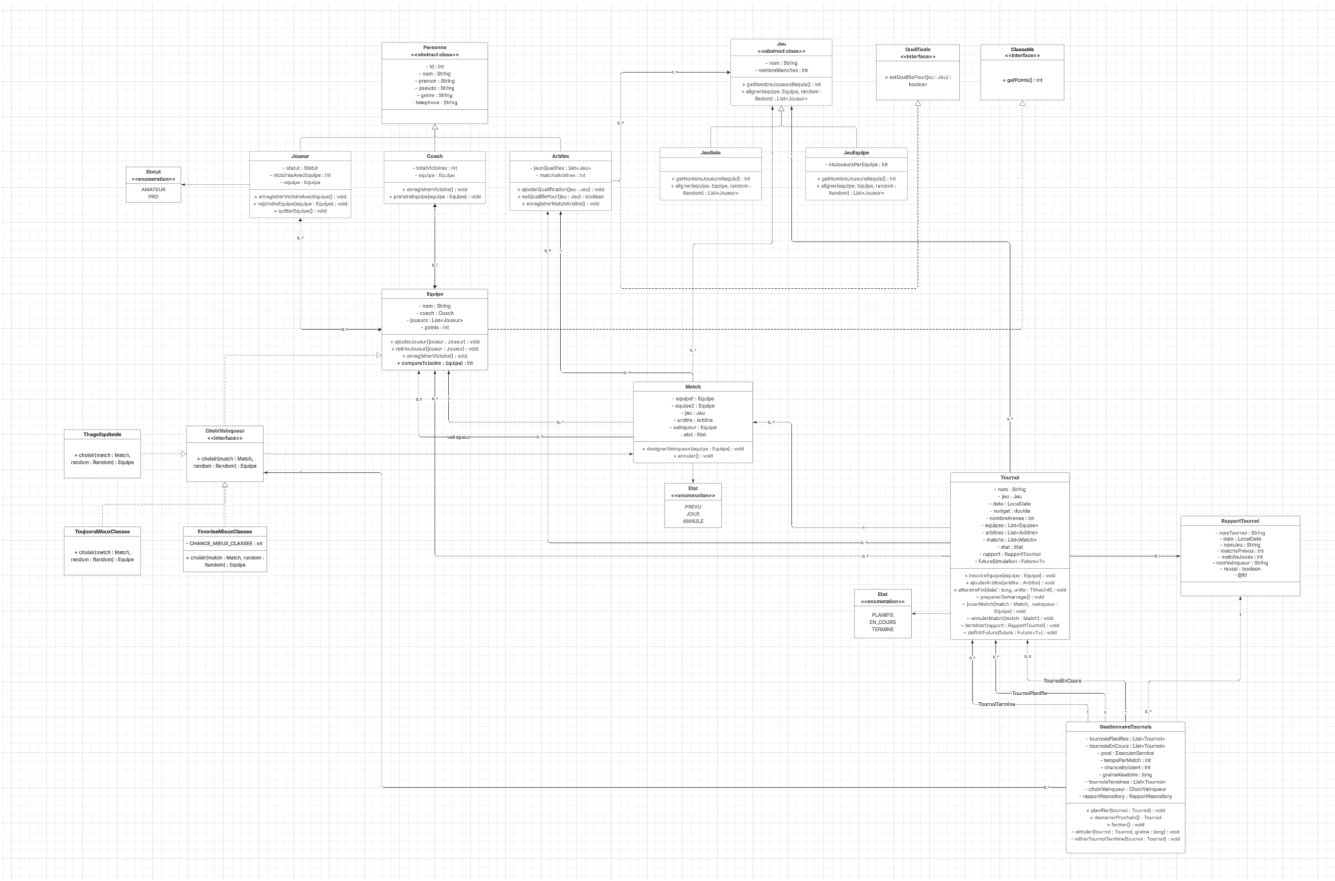



Diagramme de classes disponible dans en format .pdf dans le dossier ou en suivant ce lien :

https://lucid.app/lucidchart/94eb4ef8-a7c6-4dc7-894b-70434a4ce9ae/edit?viewport_loc=-2626%2C1666%2C4952%2C2787%2CHWEp-vi-RSFO&invitationId=inv_6cf00406-66c3-43ed-a5d5-971ae7c57c9c

# Collections utilisées

- **Listes (ArrayList).** Nous les utilisons pour les joueurs d'une équipe, les équipes, arbitres et matchs d'un tournoi, ainsi que pour les tournois en cours et terminés. Ces éléments sont surtout parcourus dans l'ordre où ils ont été ajoutés.
- **File de priorité (PriorityQueue).** Les tournois planifiés sont classés par date. Le gestionnaire peut ainsi récupérer en premier le tournoi le plus proche.
- **Ensemble (HashSet).** Les qualifications d'un arbitre sont stockées dans un ensemble de jeux. Un jeu ne peut donc être enregistré qu'une seule fois et la recherche d'une qualification est directe.
- **Table associative (HashMap).** Pendant la simulation, elle associe chaque équipe à son nombre de victoires. Cela permet de retrouver rapidement le score de chaque équipe.
- **Classement (TreeSet).** Le classement des équipes est stocké dans un TreeSet. Il les trie par points décroissants, puis par nom, et évite les doublons.
- **Recherche par nom (HashMap).** Nous utilisons un HashMap pour retrouver directement une équipe à partir de son nom.
- **Matchs par vainqueur (TreeMap).** Le TreeMap regroupe les matchs joués par équipe gagnante et les affiche selon le classement des équipes.
- **Pool de threads.** Les simulations utilisent un pool fixe de trois threads, ce qui permet de faire tourner jusqu'à trois tournois en même temps.

Avant de démarrer, le tournoi vérifie qu'il y a au moins quatre équipes, que chaque équipe a assez de joueurs pour le jeu et qu'au moins un arbitre est qualifié.

# Patterns et organisation

Nous avons utilisé le pattern Stratégie pour le choix du vainqueur. L'interface ChoixVainqueur peut être remplacée par plusieurs règles : tirage équitable, avantage probabiliste à l'équipe mieux classée ou choix toujours déterministe. Le gestionnaire peut donc changer de règle sans réécrire la simulation.

RapportRepository joue le rôle de repository : il regroupe la sauvegarde et les recherches des rapports avec JPA. Le reste du tournoi n'a pas besoin de gérer directement les requêtes à la base H2. Nous utilisons aussi l'héritage et le polymorphisme pour les types de jeu et les rôles des personnes.

# Synchronisation et exécution parallèle

Le gestionnaire synchronise les méthodes qui modifient ou consultent ses collections partagées. Les simulations s'exécutent dans un pool fixe de trois threads, ce qui limite à trois le nombre de tournois lancés en parallèle. Dans Tournoi, l'état et le rapport sont déclarés volatile pour que les changements faits par la simulation soient visibles depuis les autres threads. Un Future permet à attendreFin() d'attendre la fin d'un tournoi et de fixer un délai maximal. Enfin, le repository ouvre un EntityManager et une transaction pour chaque sauvegarde, puis ferme l'EntityManager.